# PG BUILDER

Full Project Details & Implementation Guide

**A multi-tenant SaaS platform for PG website creation, room/bed management, bookings and payments.**

Includes project scope, workflows, architecture, database design, APIs, security, testing, deployment and launch planning.

Planning blueprint • Beginner-friendly • Implementation-focused

# Contents

1. Project Overview
2. Problem Statement and Proposed Solution
3. Scope and Requirements
4. User Roles and Permissions
5. Main User Workflows
6. Website Builder and URL Strategy
7. Recommended Technology Stack
8. System Architecture
9. Database Design
10. Floors, Rooms and Beds
11. Booking Lifecycle and Double-Booking Prevention
12. Payments, Refunds and Receipts
13. Frontend Screens and Navigation
14. Backend API Design
15. File Uploads and Personal Data
16. Authentication and Security
17. Dashboard and Reports
18. Suggested Codebase Structure
19. Development Roadmap
20. Testing and Acceptance Criteria
21. Deployment and Operations
22. Business Model and Pricing Options
23. Main Risks and Decisions
24. Launch Checklist
25. Example Scenario
26. Glossary
27. Final Recommendation

# 1. Project Overview

PG Builder is a multi-tenant SaaS platform that allows multiple Paying Guest (PG) hostel owners to create and manage their own online PG websites without building separate software from scratch.

Each owner can create a PG profile, add rooms and beds, set rent and advance amounts, publish a public booking page, review customer bookings, track payments, and manage occupancy from an owner dashboard.

> **Core idea: one central PG Builder platform serves many independent PG businesses. Each PG owner sees only their own property data, while customers visit the public page for the PG they want to book.**

### Main objectives

- Let owners create a PG website using a simple setup wizard.
- Manage buildings, floors, rooms, beds, rent, rules, photos and availability.
- Allow customers to browse a PG, submit a booking request and follow payment instructions.
- Provide owner dashboards for bookings, occupancy, payments and customer records.
- Protect each owner's data through strict tenant isolation.

# 2. Problem Statement and Proposed Solution

### Problem statement

Many small PG owners manage vacancies, enquiries, tenant details, rent and advance payments using notebooks, spreadsheets, phone calls or messaging apps. This makes it difficult to keep availability accurate, find payment history, respond to enquiries quickly and maintain consistent records.

### Proposed solution

PG Builder provides a ready-made website and management dashboard. Owners configure their property once, then share a public URL with potential residents. The system keeps room/bed inventory and booking records in one place.

### Expected benefits

- Less manual work and fewer duplicate records.
- Clearer room and bed availability.
- Faster customer enquiries and booking requests.
- A central record of payments and booking status.
- A platform that can grow from one PG to many PG businesses.

# 3. Scope and Requirements

### Minimum viable product (MVP)

- Owner registration, login, logout and password reset.
- Create and edit a PG profile with name, address, contact details, photos, rules and branding.
- Configure floors, rooms, bed capacity, rent and advance.
- Public PG page showing photos, facilities, pricing and availability.
- Booking request form and booking status management.
- Payment record entry and receipt/reference tracking.
- Owner dashboard with occupancy, bookings and outstanding amounts.

- Admin controls for users, PG listings and platform health.

### Later enhancements

- Custom domains, online payment gateway, automated receipts, SMS/WhatsApp notifications, reviews, coupons, multiple staff accounts and advanced reports.

### Out of scope for the first release

A full accounting system, automatic legal compliance decisions, guaranteed tenant verification, and complex property-chain/franchise workflows should not be included until the core booking and payment processes are reliable.

## 4. User Roles and Permissions

| Role | Can do | Must not do |
|---|---|---|
| Platform Admin | Manage platform users, review listings, handle support, suspend abusive accounts, inspect platform-level metrics. | Casually change private owner data without an audit reason. |
| PG Owner | Manage their PG profile, rooms, beds, bookings, customers, prices and payment records. | Read or modify another owner's data. |
| PG Staff (later) | Perform owner-approved tasks such as viewing bookings or recording payments. | Change owner/security settings unless permission is granted. |
| Customer | Browse public PG details, submit booking requests, view their own booking/payment information. | View other customers' private details or internal owner dashboards. |

Authorization must be checked by the backend on every protected request. Hiding a button in React is not security.

## 5. Main User Workflows

### Owner setup workflow

1. Owner creates an account and verifies their email or phone.
2. Owner creates a PG profile and enters the property name, location, contact details and facilities.
3. Owner adds floors, rooms, bed capacity, monthly rent, advance amount and house rules.
4. Owner uploads approved photos and previews the public PG page.
5. Owner publishes the page and shares its public URL.
6. Owner receives booking requests and approves, rejects or follows up on them.

### Customer booking workflow

1. Customer opens a PG's public page.
2. Customer reviews facilities, rules, pricing and available beds.
3. Customer submits contact information and preferred move-in date.
4. The server checks availability and creates a pending booking or temporary hold.
5. Payment is completed or recorded according to the configured payment method.
6. The system confirms the booking only after the required verification.
7. The owner and customer can view the resulting booking status.

> **Important: a customer seeing a 'payment successful' message in the browser is not enough to confirm a payment. Verify gateway webhooks or owner-approved manual records on the server.**

# 6. Website Builder and URL Strategy

For the MVP, use a path-based URL so each PG has a unique slug. Example: pgbuilder.example/sv-pg. This is simpler to deploy and maintain than generating a separate website and deployment for every owner.

## Suggested PG profile fields

- Public name and unique slug.
- Address, map/location description, phone and enquiry email.
- Description, facilities, room types, photos and house rules.
- Rent, advance, deposit/refund policy and payment instructions.
- Publication status: draft, published, suspended or archived.

## Slug rules

Slugs should be unique, lowercase, URL-safe and validated by the server. If a slug changes, consider preserving a redirect or showing a clear not-found page for the old URL.

## Custom domains later

Custom domains require domain verification, DNS instructions, HTTPS provisioning, and a safe mapping from hostname to PG tenant. Add this only after the path-based model is stable.

# 7. Recommended Technology Stack

| Layer | Recommended choice | Purpose |
|---|---|---|
| Frontend | React | Public PG pages, owner dashboard and platform admin UI. |
| Backend | Node.js + Express | REST API, authorization, booking logic and integrations. |
| Database | MongoDB + Mongoose | Owner, PG, room, bed, booking and payment records. |
| Authentication | Secure session or short-lived access token + refresh strategy | Identify users and protect private endpoints. |
| File storage | Private object storage for identity documents; public image storage for approved PG photos | Separate public marketing images from private customer files. |
| Hosting | Frontend host + backend host + managed MongoDB | Deploy and operate the service. |
| Monitoring | Structured logs, error monitoring and uptime checks | Find failures and track service health. |

This is a practical starting point, not the only valid architecture. Keep secrets in environment variables and never commit API keys, database credentials or production tokens to source control.

# 8. System Architecture

A typical request flows from the browser to the React frontend, then to the Express API. The API authenticates the user, resolves the relevant PG tenant, validates the request, applies business rules and reads/writes MongoDB. Files are uploaded to storage rather than stored directly as large database fields.

| Component | Responsibilities |
|---|---|
| React application | Routing, forms, validation feedback, public PG display and dashboards. |

| Component | Responsibilities |
|---|---|
| Express API | Authentication, tenant authorization, data validation, booking state transitions and payment verification. |
| Tenant resolver | Maps a public slug or authenticated owner to the correct PG ID. |
| MongoDB | Stores business records with tenant ownership references and indexes. |
| Object storage | Stores PG photos and private documents with separate access rules. |
| Payment provider (optional) | Processes online payments and sends signed webhook events. |
| Email/SMS provider (optional) | Sends verification, booking and payment notifications. |

### Architecture rule

Do not trust a PG ID sent by the browser by itself. For owner operations, derive or verify the PG ID from the authenticated user's authorized membership.

## 9. Database Design

The following collection design is a starting point. In a shared database, business records should include a pgId/tenantId where relevant, and every query must be scoped to the authorized tenant.

| Collection | Important fields | Purpose |
|---|---|---|
| User | name, email, phone, passwordHash, role, status | Authentication and identity. |
| PG | ownerId, name, slug, address, status, publishedAt | Tenant/property record. |
| PGSettings | pgId, logo, theme, facilities, rules, pricing policy | Public appearance and business settings. |
| Floor | pgId, name/number, sortOrder | Building floor organization. |
| Room | pgId, floorId, roomNumber, sharingType, status | Physical room inventory. |
| Bed | pgId, roomId, bedLabel, status | Bookable bed inventory. |
| Customer | pgId, name, phone, email, privateDocumentRefs | Customer record with restricted personal data. |
| Booking | pgId, customerId, bedId, dates, priceSnapshot, status | Reservation and occupancy lifecycle. |
| Payment | pgId, bookingId, amount, currency, method, status, reference | Payment transactions and reconciliation. |
| AuditLog | pgId, actorId, action, targetId, timestamp | Important administrative changes. |

### Key relationships

- One owner can manage one or more PG records if the product allows it.
- A PG contains floors, rooms and beds.
- A booking references a customer and one or more booked beds, depending on the business rule.
- A payment references a booking and its PG.
- A booking should store a snapshot of the agreed rent, advance, dates and policy version so future price changes do not rewrite historical agreements.

### Shared database or separate database per PG?

Recommended MVP: a shared database with strict tenant scoping, unique indexes and automated authorization tests. Separate databases can provide stronger isolation for some customers, but require a central tenant registry, provisioning, migrations, backup strategy and connection management. Do not mix both approaches casually.

## 10. Floors, Rooms and Beds

Represent floors, rooms and beds as separate records when inventory needs to be managed precisely. A room can have two-sharing, three-sharing or another supported capacity; each actual bed has its own status.

### Example

| Floor | Room | Bed | Status example |
|---|---|---|---|
| 1 | 101 | A | Available |
| 1 | 101 | B | Occupied |
| 1 | 102 | A | Temporarily held |
| 1 | 102 | B | Available |

Suggested bed statuses: available, held, occupied, maintenance and inactive. Keep the status model small and define allowed transitions. A bed should not be marked available if it has a confirmed active booking.

Avoid deleting rooms or beds that have historical bookings. Prefer archiving/deactivating them so old receipts and booking records remain understandable.

## 11. Booking Lifecycle and Double-Booking Prevention

### Suggested statuses

- Requested: customer submitted a request.
- Held: the bed is temporarily reserved until holdExpiresAt.
- Pending payment: awaiting required payment.
- Confirmed: payment or owner approval requirements are satisfied.
- Rejected or cancelled: booking will not proceed.
- Checked-in: resident has moved in.
- Completed: stay has ended.
- Expired: temporary hold ended without confirmation.

### Preventing two customers from booking the same bed

1. Validate the requested bed, dates and tenant on the server.
2. Start a database transaction where supported, or use an atomic conditional update/unique active-booking constraint appropriate to the schema.
3. Create a short-lived hold with an expiry timestamp.
4. Reject any competing request if the bed is already held or booked for an overlapping period.
5. Expire stale holds with a scheduled worker and also check expiry during booking requests.
6. Confirm only after the required payment/approval checks pass.

**Never rely only on a frontend availability check. Two customers can click the same available bed at almost the same time; the backend must enforce the final decision.**

# 12. Payments, Refunds and Receipts

Support manual payment recording for the first release, then add a payment gateway when business requirements are clear. Store each payment as a separate transaction rather than overwriting a single amount field.

## Payment record fields

- PG ID, booking ID, payer/customer reference and amount.
- Currency, method (cash, bank transfer, UPI or gateway), status and transaction/reference number.
- Created time, recorded-by user, verified time and optional notes.
- Refund amount/status and gateway reference if refunds are supported.

## Payment statuses

pending, succeeded, failed, cancelled, partially_refunded and refunded. Define which statuses count toward paid totals.

## Important controls

- Use an idempotency key or provider event ID so webhook retries do not create duplicate payments.
- Verify webhook signatures on the backend.
- Do not store card numbers, CVV or other prohibited payment credentials.
- Keep an audit trail when an owner records, edits, reverses or refunds a payment.
- Generate receipts from verified payment records and label manual records accurately.

Business decision needed: does PG Builder merely record payments for each PG owner, or does it collect funds centrally and settle money to owners? Central collection introduces additional payment-provider, reconciliation, refund and regulatory considerations.

# 13. Frontend Screens and Navigation

## Public/customer screens

- Home/landing page for PG Builder.
- Public PG page with description, facilities, room types, photos, pricing, rules and contact details.
- Availability/booking form.
- Customer login or secure booking lookup.
- My bookings and payment/receipt view.

## Owner dashboard screens

- Overview: occupancy, pending requests, collections and upcoming move-ins.
- PG profile and website appearance settings.
- Floors, rooms and bed inventory.
- Bookings: filter, view, approve, reject, cancel and check-in.
- Customers and document access with restricted permissions.
- Payments, receipts, refunds and outstanding balances.

- Reports, staff access and security settings.

### Platform admin screens

- User/owner management, PG listing review, complaints/support, platform metrics and audit records.

## 14. Backend API Design

Example REST endpoints are shown below. Exact naming can change, but keep the API consistent and document request/response formats.

| Method and path | Purpose | Access |
|---|---|---|
| POST /api/auth/register | Create account | Public |
| POST /api/auth/login | Sign in | Public |
| POST /api/pgs | Create PG profile | Owner |
| GET /api/public/pgs/:slug | Read published public PG page | Public |
| PATCH /api/pgs/:pgId | Update PG profile/settings | Authorized owner |
| POST /api/pgs/:pgId/rooms | Create room | Authorized owner |
| POST /api/pgs/:pgId/beds | Create bed | Authorized owner |
| GET /api/pgs/:pgId/bookings | List tenant bookings | Authorized owner |
| POST /api/public/pgs/:slug/bookings | Submit booking request | Public with validation/rate limits |
| POST /api/bookings/:bookingId/approve | Approve booking | Authorized owner |
| POST /api/bookings/:bookingId/cancel | Cancel booking | Authorized user/owner by policy |
| POST /api/payments/manual | Record manual payment | Authorized owner |
| POST /api/payments/webhook | Receive payment provider event | Verified provider signature |
| GET /api/bookings/:bookingId/receipt | Generate or fetch receipt | Authorized user/owner |

Every route should validate inputs, return consistent error responses, enforce authorization, and avoid exposing internal stack traces or sensitive documents.

## 15. File Uploads and Personal Data

PG photos and identity documents are different types of data and should not share the same public access settings.

- Validate file type, size and file signature; do not trust the file extension alone.
- Store identity documents privately and serve them only after an authorization check, preferably through short-lived signed URLs.
- Avoid putting Aadhaar numbers, identity document URLs, phone numbers or other sensitive details in application logs.
- Collect only the personal data necessary for the stated business purpose.
- Define retention and deletion rules and explain them in the privacy notice.
- Restrict staff access to sensitive customer documents and audit every access where appropriate.

If Aadhaar or other government identity information is collected, review applicable Indian privacy and identity-data requirements before launch. Do not assume that collecting every possible document is automatically appropriate.

# 16. Authentication and Security

- Hash passwords using a suitable password-hashing algorithm; never store plaintext passwords.
- Use HTTPS in production and secure cookie/token settings.
- Enforce role-based access and tenant ownership in backend middleware/service functions.
- Validate and sanitize input; use request-size limits and rate limiting.
- Protect authentication, public booking and file-upload endpoints against abuse.
- Use environment variables or a secret manager for credentials.
- Configure CORS for known frontend origins, not every origin by default.
- Keep dependencies updated and run security checks.
- Back up the database and test restoration.
- Record audit logs for important actions such as price changes, booking cancellations, refunds and staff permission changes.

Tenant isolation is a critical requirement. A user must never be able to access another PG's records by changing a pgId or bookingId in the request.

# 17. Dashboard and Reports

## Useful owner metrics

- Total beds, available beds, held beds and occupied beds.
- Occupancy rate = occupied beds ÷ total active beds × 100.
- New booking requests, confirmed bookings, cancellations and upcoming check-ins.
- Payments received by date and method.
- Outstanding amounts and refundable balances, using clearly defined accounting rules.
- Room/bed availability and booking history.

## Reporting cautions

Define whether a report uses booking date, move-in date or payment date. Use consistent time zones and avoid counting failed/pending payments as revenue received.

## 18. Suggested Codebase Structure

```
pg-builder/
  client/
    src/
      components/
      pages/
      layouts/
      services/api.js
      hooks/
      utils/
  server/
    src/
      config/
      middleware/
      models/
      routes/
      controllers/
      services/
      validators/
      jobs/
      utils/
    uploads/   (development only; not for public sensitive files)
  docs/
    api-spec.md
    data-model.md
    deployment.md
  .env.example
  README.md
```

Keep route handlers thin. Put booking rules, payment verification and tenant checks in reusable service/middleware layers so they can be tested independently.

## 19. Development Roadmap

| Phase | Work | Completion check |
|---|---|---|
| 1. Requirements | Finalize user roles, URL approach, pricing, booking rules, payment model and privacy requirements. | Written scope and acceptance criteria approved. |
| 2. Foundation | Create repositories, environment configuration, database connection, logging and error handling. | App starts locally and health checks pass. |
| 3. Authentication | Registration, login, verification, roles and owner-to-PG association. | Protected routes reject unauthorized users. |
| 4. PG setup | Profile, settings, branding, photos, floors, rooms and beds. | Owner can publish a valid PG page. |
| 5. Public website | Slug routing, public details, pricing and availability. | Public page shows only published information. |
| 6. Booking engine | Requests, holds, approval, cancellation and conflict prevention. | Concurrent requests cannot confirm the same bed incorrectly. |
| 7. Payments | Manual records, payment status, receipts and audit trail. | Totals match verified transactions. |
| 8. Dashboards | Occupancy, bookings, customer records and reports. | Metrics reconcile with underlying records. |
| 9. Testing/security | Unit, integration, authorization, upload and end-to-end tests. | Critical acceptance tests pass. |
| 10. Deployment | Production database, secrets, HTTPS, backups, monitoring and release checklist. | Smoke tests and restore procedure verified. |

A sensible build order is to finish one vertical workflow at a time: owner creates a PG → publishes a page → customer submits a booking → owner approves it → payment is recorded → dashboard updates.

# 20. Testing and Acceptance Criteria

## Core test cases

- A new owner can register and create a PG.
- A duplicate slug is rejected with a clear message.
- A draft or suspended PG is not visible as a public listing.
- An owner cannot read or update another owner's rooms, bookings or payments.
- A booking request for an unavailable bed is rejected.
- Two simultaneous requests for one bed cannot both become confirmed for the same period.
- An expired hold no longer blocks availability.
- A payment webhook replay does not create a duplicate transaction.
- A receipt shows the correct PG, booking, amount, payment status and reference.
- Private documents cannot be downloaded by unauthenticated or unauthorized users.
- Archived beds remain linked to historical bookings.
- Backups can be restored in a test environment.

# 21. Deployment and Operations

## Production checklist

- Separate development, staging and production environments.
- Use managed database credentials and restrict network access where possible.
- Configure production frontend and backend URLs and exact CORS origins.
- Set secure environment variables in the hosting provider.
- Enable HTTPS and appropriate security headers.
- Use persistent/private object storage for uploads; do not rely on temporary server disks for important files.
- Set up database backups and test restoration.
- Monitor API errors, latency, failed jobs and payment webhook failures.
- Create a migration and rollback process for schema/application changes.
- Provide support contact details and an incident response procedure.

Example deployment options include a React frontend host, a Node/Express backend host and MongoDB Atlas. The final hosting choice should match budget, regional needs and operational experience.

# 22. Business Model and Pricing Options

PG Builder could charge owners through one of several models. Choose a simple model for the pilot and validate whether owners are willing to pay.

| Model | Advantages | Trade-offs |
|---|---|---|
| Monthly subscription | Predictable recurring revenue and easy to explain. | Owners expect ongoing support and reliability. |

| Model | Advantages | Trade-offs |
|---|---|---|
| Free tier + paid plans | Makes onboarding easier and supports upgrades. | Feature limits and support boundaries must be clear. |
| Per-booking fee | Revenue follows platform activity. | Requires accurate attribution, refunds and dispute handling. |
| Setup fee | Covers custom setup and onboarding work. | Less recurring revenue; more manual work. |

Do not decide pricing based only on development cost. Interview a few PG owners and test a small pilot before committing to complex billing.

## 23. Main Risks and Decisions

| Risk / decision | Why it matters | Mitigation |
|---|---|---|
| Tenant data leakage | Could expose customer and business information. | Central tenant authorization, automated cross-tenant tests and audit logs. |
| Double booking | Creates disputes and loss of trust. | Server-side holds, atomic updates and concurrency tests. |
| Payment mismatch | Dashboards and receipts become inaccurate. | Transaction records, webhook verification, reconciliation and idempotency. |
| Sensitive document exposure | Privacy and security impact. | Private storage, minimal collection and short-lived access links. |
| Unclear refund policy | Owners and customers may disagree. | Display policy before booking and snapshot the accepted version. |
| Scope creep | Delays launch and increases cost. | Deliver MVP first and schedule custom domains/advanced features later. |
| Separate DB per PG | Adds operational complexity. | Use shared DB initially unless a clear requirement justifies separate databases. |

## 24. Launch Checklist

- Finalize requirements, booking policy and refund/cancellation rules.
- Confirm tenant model and define the PG owner/customer permission matrix.
- Complete authentication and tenant isolation tests.
- Complete room/bed inventory and race-safe booking tests.
- Verify payment totals, receipts and audit logs.
- Review privacy notice, document handling and retention policy.
- Test mobile layouts and common low-bandwidth scenarios.
- Configure backups, monitoring, HTTPS and production secrets.
- Pilot with one or two PG owners and gather feedback.
- Fix critical issues before inviting more owners.

## 25. Example Scenario

Illustrative example only: an owner creates a PG named 'SV PG', adds a floor with Room 101, and configures two beds at ■11,000 per bed per month. The owner sets a ■20,000 advance and publishes the public page.

A customer opens the page and requests Bed A with a move-in date. The server checks the bed's status, creates a temporary hold and returns the booking reference. The owner reviews the request. After the required payment is verified or manually recorded according to the configured policy, the booking is confirmed and the dashboard updates.

> **The ■11,000 rent and ■20,000 advance are example figures only. Each owner must configure their own pricing, and the system must store the agreed amounts on each booking.**

# 26. Glossary

| Term | Meaning |
|---|---|
| SaaS | Software as a Service: customers use software hosted by the provider. |
| Multi-tenant | One application serves multiple independent customers/businesses. |
| Tenant | A PG business/property whose data must be isolated from others. |
| Slug | URL-friendly identifier such as sv-pg. |
| MVP | Minimum Viable Product: the smallest useful version to launch and test. |
| API | Interface that lets frontend and backend exchange data. |
| Webhook | A server-to-server event notification, often used for payment updates. |
| Idempotency | Repeated processing of the same operation does not create duplicate effects. |
| Audit log | A record of important actions and who performed them. |
| RBAC | Role-Based Access Control: permissions depend on a user's role. |
| Occupancy rate | Occupied beds divided by active total beds, multiplied by 100. |

# 27. Final Recommendation

Build PG Builder as a multi-tenant platform with a shared database and strict tenant scoping for the MVP. Use path-based PG URLs first, model each bed explicitly, keep booking and payment histories separate, and treat private documents carefully.

Prioritize the complete end-to-end flow over extra features: owner registration → PG setup → public page → booking request → approval/hold → verified payment record → receipt and dashboard. Once this flow is reliable, add online payment gateway support, custom domains, notifications and advanced analytics.

This PDF is a project planning and implementation guide. It is not a completed software application, and all example prices, URLs and property names are illustrative.